🧠 Linux – Base de connaissances opérationnelle (TSSR)

🔹 Informations système

Commande / Action	Pourquoi / À quoi ça sert
`cat /etc/debian_version`	Connaître rapidement la version exacte de Debian.
`uname -a`	Affiche les infos complètes du noyau et de l'OS.
`uname -r`	Connaître uniquement la version du noyau Linux.
`lsb_release -a`	Identifier distribution, version et codename.
`cat /proc/cpuinfo`	Vérifier le CPU, nombre de cœurs, architecture.
`free -m` / `free -h`	Vérifier l'état de la mémoire RAM.
`df -h`	Voir l'espace disque utilisé et disponible.
`uptime`	Vérifier le temps de fonctionnement et la charge système.

26-01-21-COMMANDE AVANCE - SCRI…

🔹 Réseau

Commande	Pourquoi / Utilité
`ip a`	Affiche toutes les interfaces réseau et leurs IP.
`ip -br a`	Vue synthétique rapide des interfaces réseau.
`ip -br -4 a`	Affiche uniquement les adresses IPv4.
`ip l`	Vérifie l'état des interfaces (UP/DOWN).
`cat /etc/resolv.conf`	Vérifie les DNS configurés sur la machine.

26-01-21-COMMANDE AVANCE - SCRI…

🔹 Processus

Commande	Pourquoi / Utilité
`ps`	Liste les processus du terminal courant.
`ps -f`	Affiche les processus avec plus de détails.
`ps -ef`	Liste **tous** les processus du système.
`ps -u user`	Voir les processus d'un utilisateur précis.
`ps -eH`	Visualiser l'arborescence des processus.

Commande	Pourquoi / Utilité
`kill PID`	Arrêter proprement un processus.
`kill -9 PID`	Forcer l'arrêt d'un processus bloqué.

26-01-21-COMMANDE AVANCE - SCRI…

◆ Redirections & pipelines

Syntaxe	Pourquoi / Utilité
>	Redirige la sortie vers un fichier (écrase).
>>	Ajoute la sortie à la fin d'un fichier.
`	`
`ls -l	grep scripts`	

26-01-21-COMMANDE AVANCE - SCRI…

◆ Scripts Bash – bases

Action	Pourquoi / Utilité
`#!/bin/bash`	Définit l'interpréteur du script.
`chmod +x script.sh`	Rend le script exécutable.
`bash script.sh`	Exécute le script sans droits d'exécution.
`./script.sh`	Lance un script exécutable.

26-01-21-COMMANDE AVANCE - SCRI…

◆ Variables & paramètres

Syntaxe	Pourquoi / Utilité
`var=valeur`	Stocker une valeur dans un script.
`$var` / `${var}`	Accéder à la valeur d'une variable.
`unset var`	Supprimer une variable.
`$0`	Nom du script.
`$1`, `$2`	Paramètres passés au script.
`$#`	Nombre de paramètres.
`$*`	Tous les paramètres.

26-01-21-COMMANDE AVANCE - SCRI…

🔹 Manipulation de fichiers (cut / awk)

Commande	Pourquoi / Utilité
`cut -d':' -f 1 fichier`	Extraire un champ précis d'un fichier.
`cut -d':' -f 2,3 fichier`	Extraire plusieurs champs.
`awk '{print $1}'`	Extraction plus flexible que `cut`.
`awk -F: '{print $4}'`	Définir un séparateur personnalisé.
`gsub()`	Remplacer une chaîne dans awk.

26-01-21-COMMANDE AVANCE - SCRI…

26-01-22- SCRIPT

🔹 Conditions (`if`)

Élément	Pourquoi / Utilité
`if [[ condition ]]`	Tester une condition dans un script.
`-f fichier`	Vérifie si un fichier existe.
`! [[ -f fichier ]]`	Tester l'absence d'un fichier.
` `	
`=, -eq, -ne, -gt…`	Comparaisons numériques ou chaînes.

26-01-22- SCRIPT

🔹 Boucles

Type	Pourquoi / Utilité
`while`	Répéter tant qu'une condition est vraie.
`for`	Boucler sur une liste ou une plage.
`break`	Sortir immédiatement d'une boucle.
`sleep`	Temporiser une boucle (ex : attente fichier).

26-01-22- SCRIPT

◆ Gestion des erreurs

Élément	Pourquoi / Utilité
`$?`	Code retour de la dernière commande.
`0`	Succès de la commande.
`!= 0`	Erreur détectée → permet une réaction conditionnelle.
`2> error.txt`	Rediriger les erreurs standard.

26-01-22- SCRIPT

◆ Gestion utilisateurs (script)

Commande	Pourquoi / Utilité
`useradd`	Créer un utilisateur de manière non interactive.
`chpasswd`	Définir le mot de passe d'un utilisateur.
`usermod -aG sudo`	Ajouter un utilisateur au groupe sudo.
`userdel -r`	Supprimer un utilisateur et son répertoire.